

Computer Programming

Lecture 2

C/C++ Pointers

Class Activity - 01

- Write a program that generates the following output, using only one *cout statement* and *proper spacing*

1900	2135
1950	2235
2000	2335
2050	2435

Why we need pointers?

- **Why we need Pointers:**
 - Accessing array elements
 - Passing arrays and strings as arguments
 - Creating data structures such as linked lists
 - Dynamic memory allocation, which can grow at runtime using.

Lecture Contents

- Introduction to Pointers
- Pointer variable declaration
- Pointer initialisation
- Referencing
- Dereferencing
- Arithmetic on Pointers

Regular Variables

- Variables are used to keep track of data in the computer's memory.
- Declaring a regular variable = Allocating a location in memory to store a value

- **Example:**

```
int myInt = 0; //Allocates enough space in memory  
              //to store an int and puts 0 there
```

- The location in memory has an address
- Use *address-of* operator & to get the address

- **Example:**

- `cout << "Address of myInt = " << &myInt << endl;`

Pointer Variables

- **A pointer variable is used to hold an address**

- Use operator `*` to declare a pointer variable

- Example:

```
int *pMyInt; //Declares an empty pointer
```

- An address must be assigned to the pointer variable before it can be used

```
int myInt = 0;    //Allocates memory, stores 0
```

```
int *pMyInt;      //Declares an empty pointer
```

```
pMyInt = &myInt; //Puts an address in the pointer
```

Pointer Variable

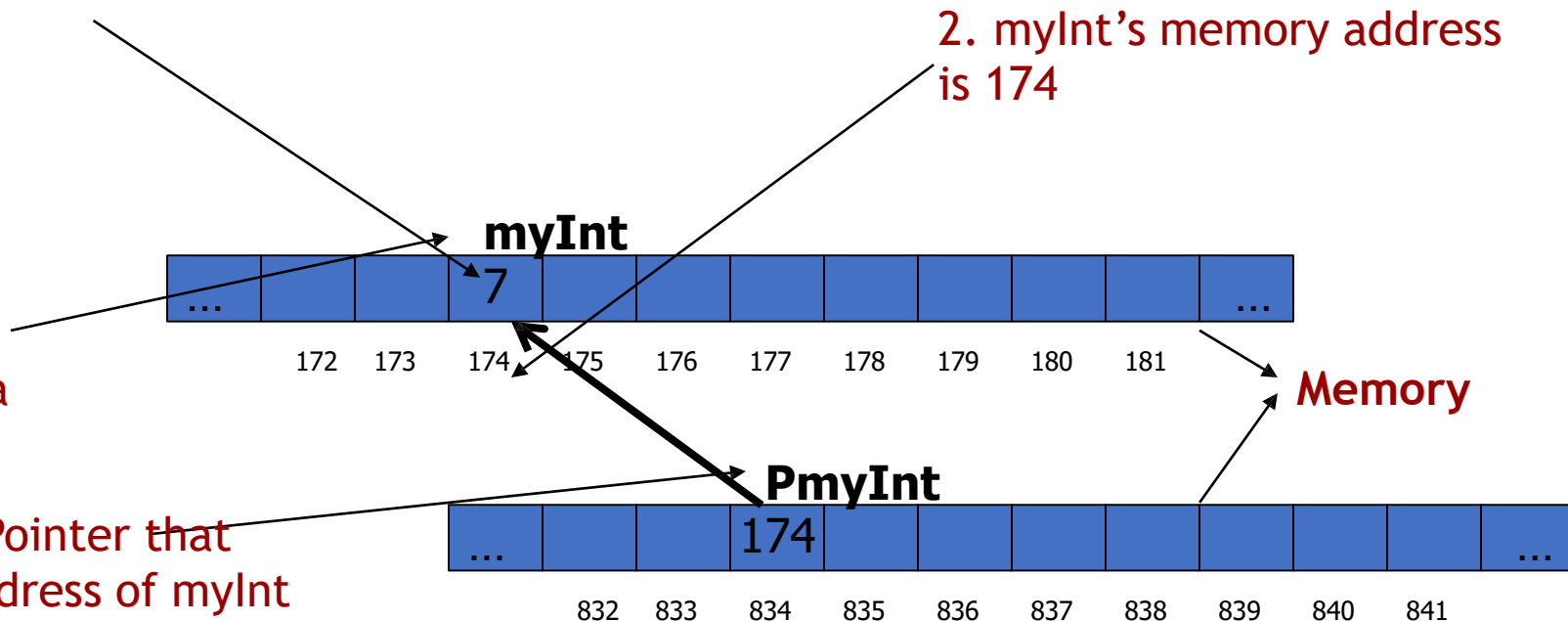
- The main difference between normal variables and pointers is that, **variables contain a value**, while **pointers contain a memory address**.
- **Memory address:**
 - We know each memory address contain some value.
 - Thus if pointers contain memory addresses, we can get its value (indirectly).

3. Content at the memory address 174 is 7

2. myInt's memory address is 174

1. myInt is a variable

PmyInt is a Pointer that holds the address of myInt



Dereferencing a Pointer

- Accessing the object addressed by the pointer
- Dereference Operator `*` used with the name of the pointer, *after it is declared and initialized*
- Examples

```
int myInt = 0;      //Allocates memory, stores 0
int *pMyInt;        //Declares an empty pointer
pMyInt = &myInt;    //Puts address in the pointer

cout << *pMyInt << endl; //Prints 0
*pMyInt = 5;         //puts 5 into myInt
cout << myInt << endl;   //Prints 5
cout << *pMyInt << endl; //Also prints 5
cout << pMyInt << endl;  //What prints?
```


Where are the errors?

```
int main() {  
  
    int m;  
    int *pm;  
    *pm = 5;  
  
    int n;  
    int *pn = &n;  
    pn = 5;  
  
}
```

Where are the errors?

```
int main() {
```

```
    int m;
```

```
    int *pm;
```

```
    *pm = 5;
```

ERROR! No address in pm

//Correction

pm = &m;

*pm = 5;

```
    int n;
```

```
    int *pn = &n;
```

```
    pn = 5;
```

```
}
```

Where are the errors?

```
int main() {
```

```
int m;  
int *pm;  
*pm = 5;
```

ERROR! No address in pm

//Correction
pm = &m;
*pm = 5;

```
int n;  
int *pn = &n;  
pn = 5;
```

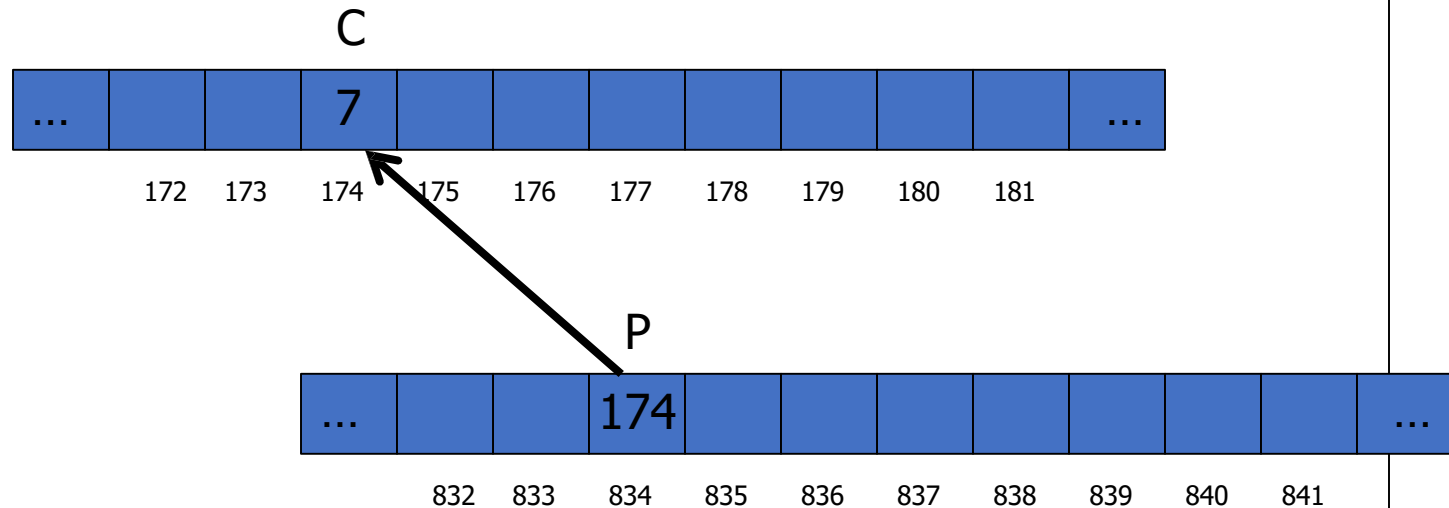
ERROR! Missing operator*

//Correction
*pn = 5;

```
}
```

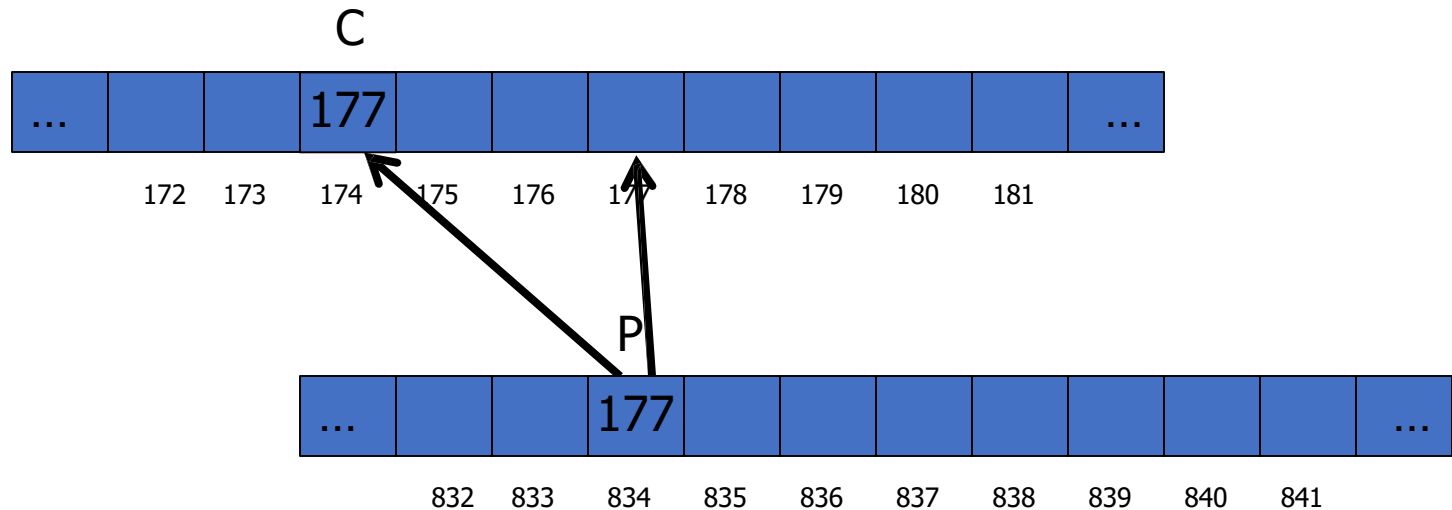
Dereferencing—Another Example

```
int C;  
int *P; /* Declare P as a pointer to int */  
C = 7;  
P = &C;  
cout<<*p<<endl;    //What is the output?
```



Dereferencing

```
cout << *P; /* Prints out '7' */  
*P = 177;  
P = 177; /* This is unadvisable!! */
```

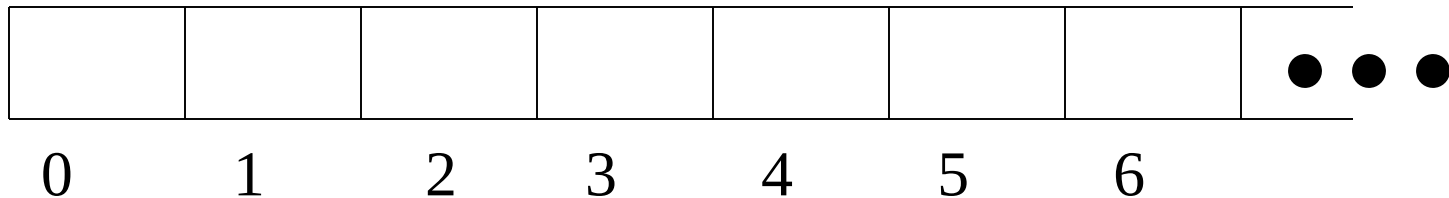


Pointers--Review

- All data (software instructions, variables, arrays) is in memory.
- Each memory location has an address.
- A **pointer** is a C version of the address.
 - * denotes we are accessing or storing the value in the address of the pointer.
denotes we are accessing or storing the memory address in the pointer.
- If any pointer knows the address of any memory address. It can easily change its value
- Always remember Only 0 and NULL evaluate to FALSE.

Bits and Bytes

- Memory is divided into bytes, each of which are further divided into bits
 - Each bit can have a value of 0 or 1
 - A byte is eight bits
 - Can hold any value from 0 to 255
- Memory can be thought of as a sequence of bytes, numbered starting at 0

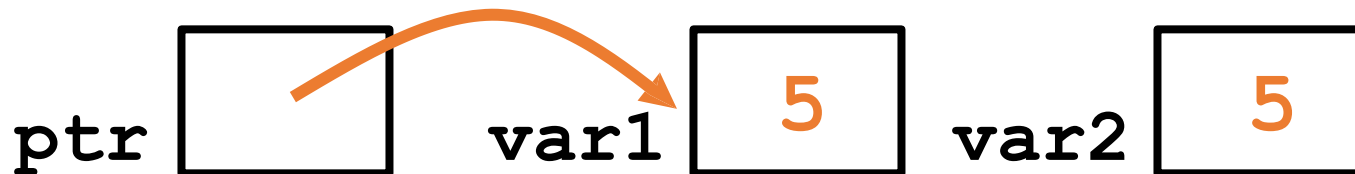


Storing Variables

- Each variables is stored in some sequence of bytes
 - Number of bytes depends on the data type
 - Two variables will never share a byte – a variable uses all of a byte, or none of it
- **Example:**
 - An int is usually stored as a sequence of **four bytes**

Pointers & Allocation

- Pointing to something that already exists:
 - `int *ptr, var1, var2;`
 - `var1 = 5;`
 - `ptr = &var1;`
 - `var2 = *ptr;`
- `var1` and `var2` have room implicitly allocated for them.



Arithmetic on Pointers (1/4)

- A pointer may be incremented or decremented.
- This means **only address** in the pointer is incremented or decremented.
- An integer (**can be constant**) may be added to or subtracted from a pointer.
- Pointer variables may be subtracted from one another.
- Pointer variables can be used in comparisons, but usually only in a **comparison to pointer variables or NULL**.

Arithmetic on Pointers (2/4)

- When an integer (**n**) is added to or subtracted from a pointer (**ptr**)
 - The new pointer value (**ptr**) is changed by the **ptr** address **plus (+)** **n** **multiple (*)** the (bytes of **ptr** data type).
 - **ptr + n * (bytes of ptr data type)**
- Example
 - `int *ptr;`
 - `ptr = 1000; // Assume that 1000 is a memory address. e.g., &n`
 - `ptr = ptr + 2; // moves pointer two slots, where each slot is of 4 bytes`
 - `// ptr address is now changed to 1000 + 2*4 (because integer consumes 4 bytes) New address is now 1008`
 - `cout<<ptr; // displays 1008`

Arithmetic on Pointers (3/4)

- Example (2)

- `long *ptr;`
- `ptr = 1000;`
- `ptr++;` **// $1000 + 1*4 = 1004$**

- Example (3)

- `float *ptr;`
- `ptr = 1000;`
- `ptr+=3;` **$1000 + 3*4 = 1012$**

- Example (4)

- `int *ptr;`
- `int num1 = 0;`
- `ptr = &num1;`
- `ptr++;` **// $1002 + 4 = 1006$**

Address	data
1000	
1002	num1
1004	
1006	
1008	

Memory width is 2 bytes

Arithmetic on Pointers (4/4)

- Example (5)
 - void main (void)
 - {
 - int *pointer1, *pointer2;
 - int num1 = 93;
 - pointer1 = &num1; //address of num1
 - pointer2 = pointer1; // pointer1 address is assigned to pointer2
 - cout<<"Address stored in pointer1"
 - cout<<pointer1;
 - cout<<"Address stored in pointer2"
 - cout<<pointer2;
 - }

Address	data
1000	
1002	num1 = 93
1004	
1006	pointer1 = 1002
1008	pointer2 = 1002

Logical operators on Pointers

- We can apply logical operators (<, >, <=, >=, ==, !=) on pointers.
 - But remember **pointers can be compared to pointers** or
 - **NULL**
- Example (6)
 - `int *pointer1, *pointer2;`
 - `int num1 = 93;`
 - `If ( pointer1 == NULL ) // pointer compared to NULL`
 - `pointer1 = &num1;`
 - `pointer2 = &num1;`
 - `If ( pointer1 == pointer2 ) // pointer compared to pointer`
 - `printf("Both pointers are equal");`

- Some Examples (CodeProject)
- <http://www.codeproject.com/Articles/11560/Pointers-Usage-in-C-Beginners-to-Advanced>

- Questions?